Тестовое задание — стажёр (Intern QA)

Общие правила

- Задание выполняется самостоятельно, без сторонней помощи и LLM.
- Ответы оформляются в одном документе (Markdown), каждая секция подписана. Окончательный документ выложен в github/lab.
- Для заданий 2–4 обязательно поясняйте **ход мыслей**: какие техники тест-дизайна использовали и почему именно они.
- Время на выполнение: **7 дней**.
- **Важно:** не описывайте «общие» тест-кейсы. Ваши случаи должны вытекать из конкретных данных и условий, данных в задаче. За каждое необоснованное допущение — минус.

Задание 1. «Будильник» — анализ требований и тест-дизайн

Контекст

Разрабатывается мобильное приложение «Будильник». Продуктовая команда описала требования:

> **Требование 1.** Пользователь может создать будильник, указав **время** (ЧЧ:ММ) и **дни недели** (один или несколько, включая вариант «каждый день»).
>
> **Требование 2.** Будильник может быть **одноразовым** (срабатывает один раз в указанную дату) или **повторяющимся** (срабатывает каждые выбранные дни недели).
>
> **Требование 3.** Когда будильник срабатывает, приложение показывает уведомление. Пользователь может **отложить** (snooze) на N минут или **остановить** (stop).
>
> **Требование 4.** Одноразовый будильник после срабатывания **автоматически отключается**.
>
> **Требование 5.** Пользователь может увидеть список всех своих будильников и **включить/выключить** любой без удаления.

Подзадачи

1. **Найдите пробелы и противоречия.** Составьте список вопросов к продукт-менеджеру (минимум 5) — то, что в требованиях не определено и влияет на реализацию.
2. **Составьте приоритизированный чек-лист** (smoke-критерии) для быстрой проверки базовой работоспособности — не больше 8 пунктов.
3. **Примените минимум 3 техники тест-дизайна** на выбор (классы эквивалентности, граничные значения, таблица решений, попарное тестирование, диаграмма состояний, use-case) и распишите, **какой сценарий вы бы тестировали каждой техникой**.
4. **Бонус вопрос: Опишите один особо рискованный сценарий**, который легко пропустить (edge case), и объясните почему.

> **Учтите**: время вводится в формате 24 часа, есть переход на летнее/зимнее время, есть зона «долгого сна», где snooze повторяется до 5 раз.

Пример ответа «хорошего уровня» на бонусный вопрос Задания 1

> **Рискованный сценарий:** будильник установлен на 02:30 в день перехода на летнее время, когда часы прыгают с 02:00 на 03:00. В этот день времени 02:30 **не существует**. Если приложение не обрабатывает несуществующие времена, будильник либо не сработает, либо сработает «на ровном месте» через год. Ожидается чёткое правило: пропустить этот день или перенести срабатывание.

Это эталон уровня ответа — ваши формулировки должны быть не менее конкретными.

Задание 2. «Создание заказа» — проверка API и обработки ошибок

Контекст

Интернет-магазин предоставляет API для **создания заказа**. Клиент отправляет HTTP-запрос на **POST /orders**.

Пример запроса:
```
{
  "user_id": 15,
  "items": [
    {
      "product_id": 101,
      "quantity": 2
    }
  ]
}
```

> **Требование 1.** Для **создания заказа** необходимо передать `user_id` и непустой список `items`.
>
> **Требование 2.** Каждый товар должен содержать `product_id` и `quantity`.
>
> **Требование 3.** Количество товара `(quantity)` должно быть **положительным целым числом**.
>
> **Требование 4.** Если **все данные корректны** и **товары доступны в нужном количестве**, система создаёт заказ и возвращает *HTTP 201 Created*, идентификатор заказа и его статус `created`.
>
> **Требование 5.** Если запрос **содержит некорректные данные**, система возвращает *HTTP 400 Bad Request*.
>
> **Требование 6.** Если **товар не существует**, система возвращает *HTTP 404 Not Found*.
>
> **Требование 7.** Если **товара недостаточно на складе**, система возвращает *HTTP 409 Conflict*.
>
> **Требование 8.** Если **сервис проверки остатков временно недоступен**, система возвращает *HTTP 503 Service Unavailable*.

Подзадачи

1. **Анализ требований**.
Найдите минимум 3 неоднозначности или неописанных условия, которые могут повлиять на реализацию или тестирование. Сформулируйте вопросы к аналитику или разработчику.

2. **Чек-лист тестирования.**
Составьте чек-лист из 8-12 проверок создания заказа. Обязательно включите позитивные и негативные сценарии. Каждая проверка должна быть привязана к конкретному требованию.
3. **Тест-кейсы API.**
Подготовьте 3 подробных тест-кейса. Для каждого укажите:
- предусловия;
- HTTP-метод и endpoint;
- тело запроса;
- ожидаемый HTTP-статус и результат.
> Один тест-кейс должен проверять успешное создание заказа, второй — некорректные данные, третий — одну из ошибок взаимодействия с зависимым сервисом.
4. **Проверка результата**.
Объясните, почему недостаточно проверить только HTTP-статус ответа. Что ещё нужно проверить, чтобы убедиться, что заказ действительно создан корректно?
5. **Бонус вопрос**.
Клиент отправил запрос на создание заказа, но соединение оборвалось до получения ответа. Клиент не знает, создался заказ или нет, и отправляет тот же запрос повторно. Какие проблемы могут возникнуть? Какие вопросы вы зададите команде и что предложите проверить?

Задание 3. SQL — базовые запросы

Структура таблиц

`users`

id	name	city	created_at
1	Анна	Москва	2024-01-10
2	Борис	СПб	2024-02-15
3	Виктор	Москва	2023-12-01
4	Глеб	Казань	2024-05-20

`orders`

id	user_id	total	status	created_at
101	1	3000	completed	2024-03-01
102	1	700	cancelled	2024-03-05
103	2	2500	completed	2024-03-10
104	3	1500	completed	2024-02-01
105	3	1200	returned	2024-03-15

> Статусы: `completed` (оплачен), `cancelled` (отменён), `returned` (возвращён). Оплаченным заказом считается только `completed`.

Задания (напишите SQL)

1. Вывести список пользователей **из Москвы**.
2. Вывести **общую сумму оплаченных заказов** по каждому пользователю (включая пользователей без заказов).
3. Вывести пользователей, у которых **есть хотя бы один заказ со статусом `returned`**.
4. Вывести **топ-3 пользователей по сумме оплаченных заказов**.
5. Вывести пользователей, **не делавших ни одного заказа**.

6. *(дополнительно, на оценку)* Написать запрос, который выводит для каждого города **среднюю сумму одного заказа** по оплаченным заказам.

> Учитывайте возможные особенности: `LEFT JOIN` для пользователей без заказов, агрегация с `GROUP BY`, фильтрация где-то `WHERE`, а где-то `HAVING`.

Критерии оценки

Критерий	Что смотрим	Вес
Мышление / анализ требований	Найдены реальные пробелы, а не «общие места»	30%
Техники тест-дизайна	Корректное применение, осознанный выбор, а не «просто перечислил»	30%
SQL	Правильные JOIN, агрегации, учёт краевых случаев	25%
Оформление и обоснование	Логика рассуждений, ссылки на данные задачи	15%
